IE 678: Deep Learning

Assignment No. 2 (FSS 2026)

Prof. Dr.-Ing. Margret Keuper

University of Mannheim
Chair of Machine Learning

Task	Maximum	Points
1	5	.
2	20	.
3	10	.
4	9	.
Total	44	.

Rules for Submission

a) Hand-in your solution via ILIAS until the date specified there. This is a hard deadline.

b) Pair submission:

   (i) We strongly recommend to work on this assignment in pairs—i.e, with one (and only one) additional student—and then hand in a pair submission.

   (ii) Both students must submit exactly the same contents.

c) Code:

   (i) You must submit all relevant code files (Python scripts etc.) that we require to run your code and to replicate your results.

   (ii) In particular: You must submit all Python scripts that we provided to you.

   (iii) You must include a PDF of every Jupyter notebook in which the most relevant cells are executed (i.e., make sure to disable excessively verbose cells that inflate the page count).

   (iv) It suffices to fill out the "holes" that are marked in the code fragments.

d) Non-code parts of assignment:

   (i) You must provide your answers by filling out the provided solution LaTeX file.

   (ii) You must submit a single PDF file (do not submit LaTeX source code). All answers must be provided *inside* the boxes (i.e., no resizing of boxes, no overflow, no additional pages, no modification of font (size) etc.) specified therein.

   (iii) You may add any LaTeX packages that you absolutely need.

   (iv) You must fill out the AI declaration at the end of the solution file following the provided instructions.

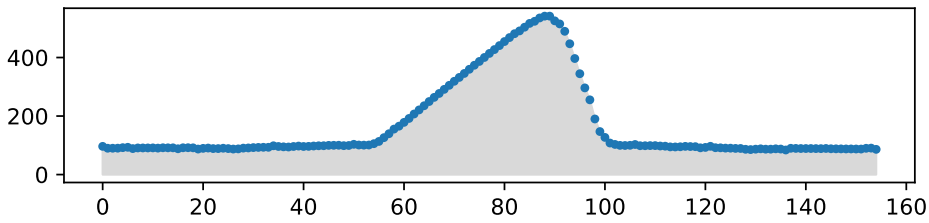
   (v) The solution PDF and all notebooks must clearly report the name and matriculation number of both students right at the beginning.

   (vi) Name your solution PDF as follows:
   `a0[assignment number]-[ilias name 1]-[ilias name 2].pdf`
   For instance: `a01-gmayer-tswift.pdf`

We reserve the right to refuse to grade any assignment that violates any of these rules.

Task 1: Climbing with a CNN — 5 Points

Recall the example of the lecture 05 (Part Embeddings), where we computed the total number of meters climbed up in a bike ride from Mannheim to Weißer Stein and back.



Throughout this assignment, we are going to use PyTorch's pre-canned layer implementations (see `https://pytorch.org/docs/stable/nn.html`). In this warm-up task, you will solve the above problem with a hand-crafted convolutional neural network (CNN).

a) Finish the implementation of the CNN in the class `ClimbCNN` in `a02_functions.py`. Use only:

   (i) A one-dimensional convolutional layer (see `https://pytorch.org/docs/stable/generated/torch.nn.Conv1d.html`), added to the model in the constructor and used in `forward()`.

   (ii) A suitable activation function as a layer (see `https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity`), added to the model in the constructor and used in `forward()`.

   (iii) a suitable readout function, used in `forward()`. (3 points)

b) Modify the model's hyperparameters and parameters such that it *additionally* computes the total number of meters climbed down. (2 points)

Task 2: MNIST-1D — 20 Points

As you may know, a classical dataset in machine learning is MNIST,[1] a collection of 60 000 handwritten digits (0–9). Since even simple feedforward neural networks nowadays achieve near zero classification error, researchers have developed alternatives such as MNIST-1D,[2] which are more difficult, while still being lightweight and flexible.

As the name suggests, MNIST-1D is inspired by MNIST. It therefore builds upon templates of length 40 that, when rotated, resemble hand-written digits (see Figure 1). These templates are processed further (using transformations, adding noise, etc.) to obtain a dataset of 5 000 items.

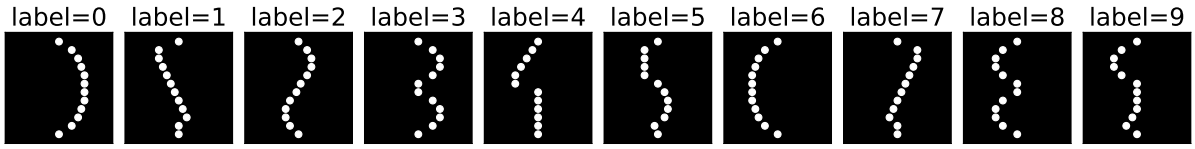


Figure 1: Template "digits" of MNIST-1D

a) Study MNIST-1D's `https://github.com/greydanus/mnist1d/blob/master/mnist1d/data.py`. **Briefly explain** in your own words how the templates are used to generate a dataset instance. (3 points)

b) Complete the implementation of `SimpleCNN` in `a02_functions.py`. Use the following architecture

$$\begin{aligned} \text{model:} &\quad \texttt{conv-block} \to \texttt{conv-block} \to \texttt{linear} \\ \texttt{conv-block:} &\quad \texttt{conv} \to \texttt{relu} \to \texttt{conv} \to \texttt{relu} \to \texttt{maxpool} \end{aligned}$$

The model has the following hyperparameters:

- `channels`: number of output channels in each `conv` layer
- `kernel_size`: kernel size used in each `conv` layer
- `stride`: stride used in each `conv` layer
- `padding`: amount of zero-padding performed in each `conv` layer

Each (spatial) max-pooling layer should use a kernel size of 2 and a stride of 2.

The final linear layer should output a 10-dimensional real-valued vector. The number of inputs of this layer is given by `linear_in`, a value that you should compute and set by hand later.

(4 points)

c) **Compute** *by hand* the number of parameters (i) in each layer and (ii) overall when:

- `channels` = 5
- `kernel_size` = 3
- `stride` = 2
- `padding` = 1

Your solution must **include** a detailed derivation of your result. Verify your result using the function `count_model_params` provided by us. (3 points)

[1] See `https://en.wikipedia.org/wiki/MNIST_database` for instance.
[2] See `https://arxiv.org/abs/2011.14439`.

d) Complete the implementation of the training loop in the function `train_model` in the script `a02_functions.py`. We are going to use PyTorch's dataset facilities and loss functions to do so.

(i) Study `https://pytorch.org/docs/stable/data.html`. **Explain** why for this task a map-style dataset is more suitable than an iterable-style dataset for working with the MNIST1D data.

(ii) Study the source code of `TensorDataset` `https://github.com/pytorch/pytorch/blob/v2.6.0/torch/utils/data/dataset.py#L193` and **explain** whether a `TensorDataset` is a map-style or iterable-style dataset.

Then, use the training items `x_train` and their labels `y_train` to create a `TensorDataset`.

(iii) Study the documentation on loss functions at `https://pytorch.org/docs/stable/nn.html#loss-functions`. **Explain** why using `CrossEntropyLoss` is more suitable than using `NLLLoss` for your CNN.

Then, define your `loss_fn` and choose an `optimizer` of your preference (e.g., Adam).

(iv) Conduct the forward pass using the batched items `x` and labels `y` and store the obtained loss in the `results` dictionary.

(v) Then, conduct the backward pass by computing the gradients of the loss with respect to all learnable parameters and performing the gradient descent step. As discussed in the tutorial, zero-out the gradients after each step.

(vi) Finally, train and evaluate `SimpleCNN` using the default hyperparameters for the model and the training.

Note. You should arrive at a test accuracy of around $90\% \pm 5\%$. If you are far off that range, revise your implementation before proceeding with the task. (5 points)

The provided model `SimpleMLP` is a feedforward neural network with three hidden layers of size Z.

e) **Compute** the number of parameters of `SimpleMLP` by hand when $Z = 256$. Verify your result using the function `count_model_params` provided by us.

Based on this result (and before proceeding with the task!), **conjecture** how `SimpleMLP` is going to perform in comparison to `SimpleCNN`. (2 points)

f) Train and study `SimpleMLP`'s performance using the default hyperparameters. **Discuss** the results with respect to your prior conjecture. (1 point)

The "one-dimensional digits" of MNIST-1D can be shuffled along the spatial dimension using the function `shuffle_pixels`.

g) Retrain both models `SimpleCNN` and `SimpleMLP` on the shuffled dataset.

h) **Conjecture** how `SimpleCNN` and `SimpleMLP` will perform in comparison to one another on the shuffled dataset. (1 point)

i) Evaluate both models and **discuss** the results with respect to your conjectures. (1 point)

Task 3: Visual Analysis — 10 Points

The notebook for this task contains some code that showcases how to produce several visualizations. Your goal is to acquire a deeper understanding of the inner workings of your CNN. For that, start by following the given subtasks. In your report, **discuss** all findings that you make during this task.

a) Start by training your CNN.

b) Extract the embeddings of the training data and test data after the last `conv-block` of your trained model (i.e., before the linear prediction head). (1 point)

c) **Conjecture** how the t-SNE plots between the (i) input data, (ii) the training embeddings, and (iii) the test embeddings might differ from one another. (3 points)

d) Following the example for the input data, produce t-SNE plots for both the training and test embeddings.

   Compare the two t-SNE plots and **discuss** your findings, also with respect to your conjectures from before.

   Hint. As always, it might be helpful to tweak the perplexity of t-SNE. (2 points)

e) Repeat the preceding subtasks but extract and use embeddings of different layers. **Discuss** your findings. (2 points)

f) Recall that in the lecture, kernels in CNNs were also described as "feature detectors". Depending on the class of the input, different features of different layers might be relevant for the classification task. **Investigate** this using the provided method `examples_heatmap` and **report** your findings. (2 points)

Task 4: Exploration 9 Points

This task is about tweaking your CNN to obtain insights on the effect of its hyperparameters on its performance and general behavior.

Task. Study *individually* the consequences (accuracies, no. of parameters etc.) of changing at least:

a) Learning rate (`lr`), batch size (`batch_size`), and/or the number of epochs (`epochs`) (recall lecture 03-2)

b) Convolutional kernel size (`kernel_size`)

c) Number of channels (`channels`)

d) Stride (`stride`)

Your submission must include and discuss at least two different hyperparameter choices for each subtask (i.e., 8 different setups in total). **Report and discuss** your results, **use visualizations** as suitable, and **draw conclusions** as to which design appears the most suitable. Also **include** a table that summarizes your results.

Optional. You are free to use an HPO library such as `https://optuna.readthedocs.io/en/stable/` or `https://docs.ray.io/en/latest/tune/index.html` (not included in the environments provided by us). (9 points)